Дополнительные материалы

В дополнительных материалах мы:

- рассмотрим понятие валидационной выборки, а также еще один несложный инструмент регуляризации;
- с нуля разработаем нейросеть с оптимизатором Adam (и таким образом создадим такую же сеть, как на последнем занятии вводного курса, но без использования Keras);
- подробнее изучим различные особенности реализации нейросети в библиотеке Keras.

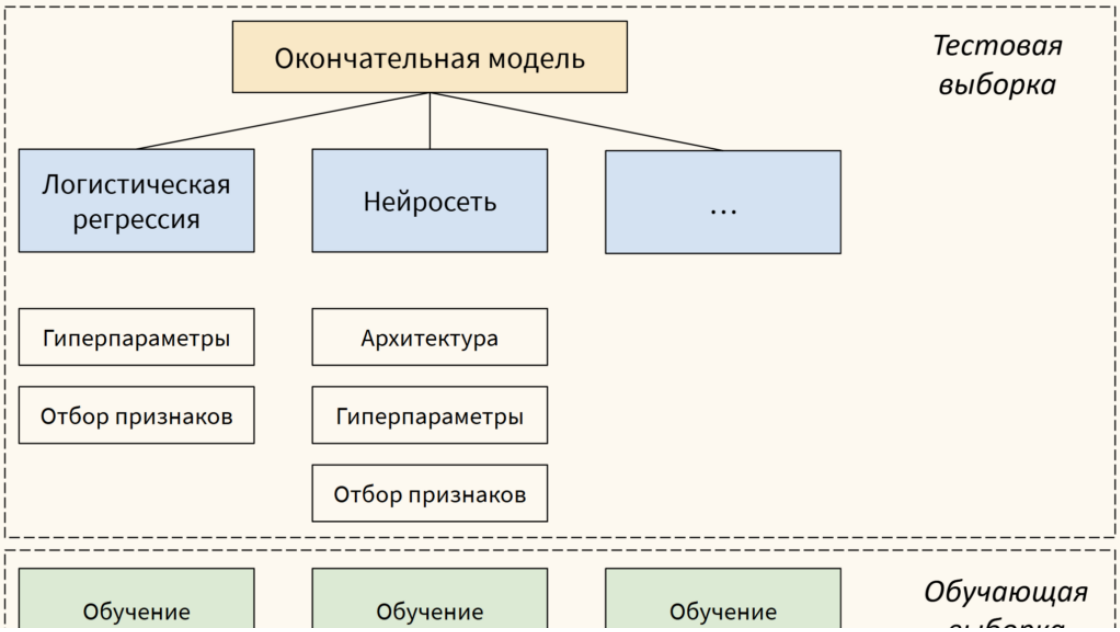
Примечание. Сегодня мы в большей степени будем сосредоточены на техническом аспекте создания и обучения моделей, а не на поиске оптимальных гиперпараметров и параметров.

Полезное введение

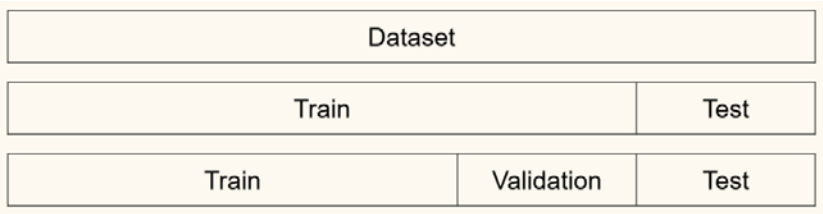
Обучающая, валидационная и тестовая выборки

В процессе решения задачи машинного обучения помимо непосредственного обучения модели необходимо выбрать алгоритм, подобрать архитектуру (в случае нейросети), настроить гиперпараметры, отобрать или создать новые признаки и т.д.

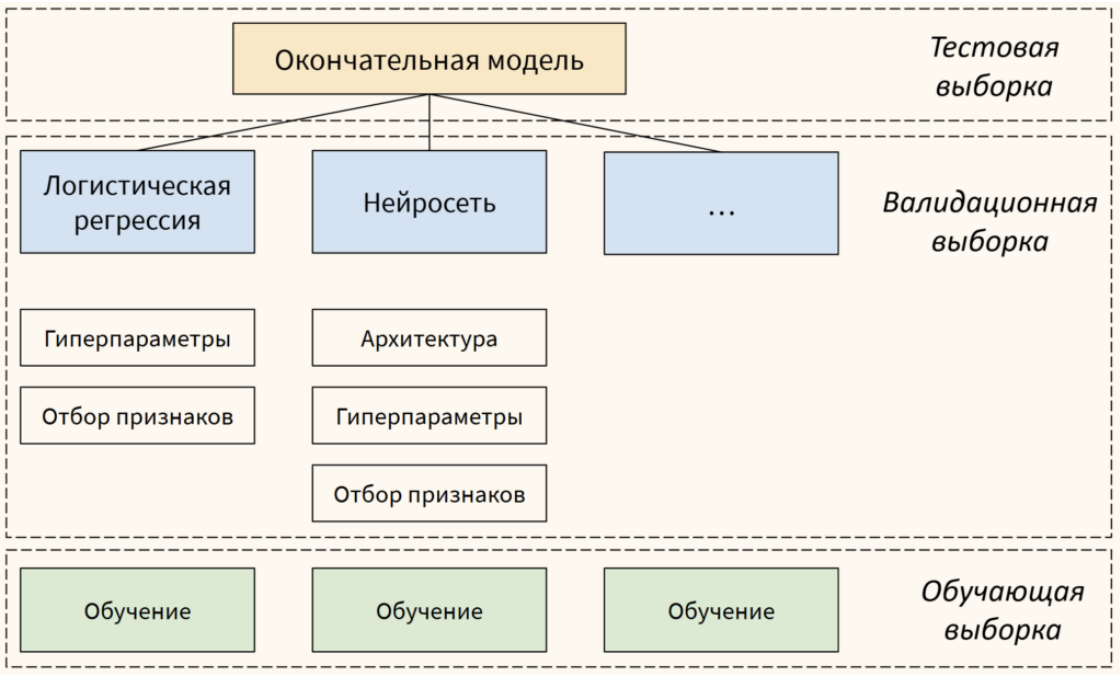
Если мы будем производить оценку упомянутых вариантов с помощью тестовых данных, то модель выучит их и, таким образом, ухудшит свою обобщающую способность.



Решение заключается в том, чтобы разделить данные на три части: обучающую (train), валидационную (validation) и тестовую (test) выборки.



Вначале на обучающем и валидационном множествах выбирается алгоритм, выполяется поиск гиперпараметров и работа с признаками. Затем на тестовых данных оценивается качество финальной модели.



Тестовые данные в этом случае, поскольку модель их не видит в процессе обучения и настройки, еще называют hold-out или out-of-sample данными.

Технически такое разделение можно выполнить, например, дважды применив функцию **train_test_split()**.

Откроем новый блокнот

Подготовим данные.

```
X, y = np.arange(2000).reshape((1000, 2)), np.arange(1000)
X.shape, y.shape
```

```
((1000, 2), (1000,))
```

Зададим пропорции обучающей r_{train}, валидационной r_{val} и тестовой r_{test} выборки.

```
r_train, r_val, r_test = 0.80, 0.10, 0.10
```

Разделим изначальные данные сначала на обучающее и тестовое множество, а затем из первой части выделим валидационную выборку.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    test_size = r_test)

r_val_star = r_val / (r_train + r_val)
X_train, X_val, y_train, y_val = train_test_split(X_train, y_train,
                                                  test_size = r_val_star)

y_train.shape, y_val.shape, y_test.shape
```

```
((800,), (100,), (100,))
```

Для сохранения исходных пропорций долю валидационных данных при втором применении функции r^*_{val} можно рассчитать по формуле

$$r^*_{val} = \frac{r_{val}}{r_{train} + r_{val}}$$

В примере выше

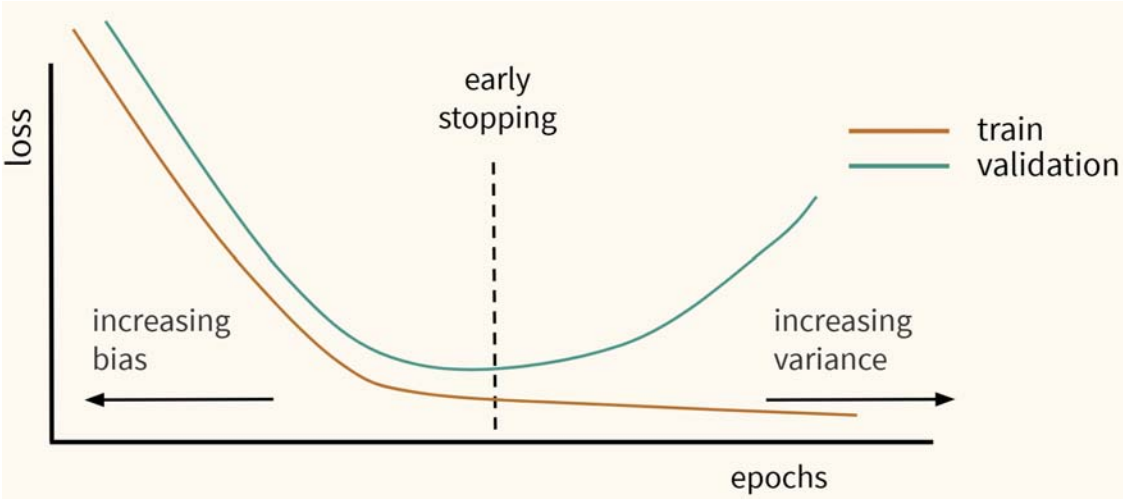
```
r_val / (r_train + r_val)
```

```
0.11111111111111112
```

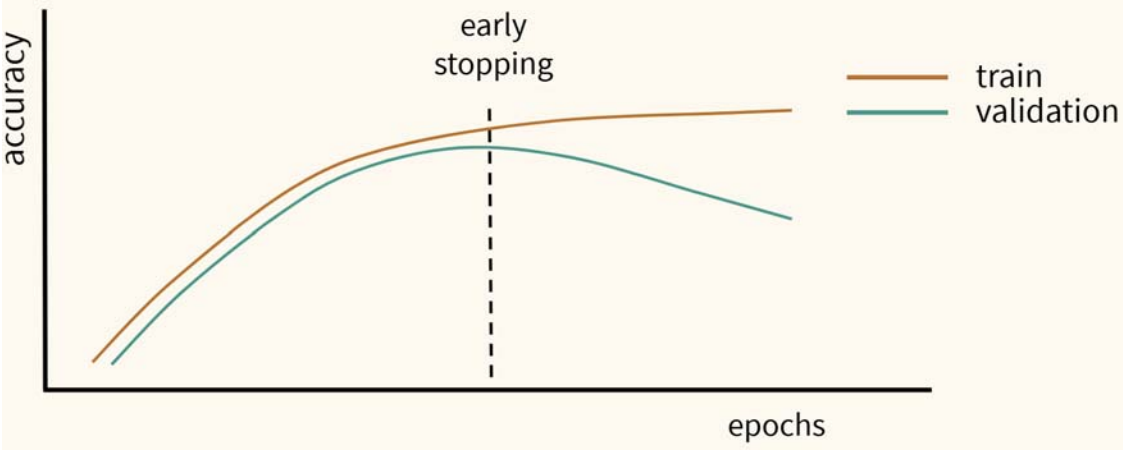
Ранняя остановка обучения

Одним из способом борьбы с переобучением является **ранняя остановка** (early stopping) обучения. Идея заключается в том, чтобы в процессе подбора параметров «отслеживать» уровень ошибки или какую-либо метрику качества (например, accuracy) не только на обучающей, но и на валидационной выборках.

Если в процессе обучения при снижающейся на «трейне» ошибке или растущей accuracy эти показатели на валидационных данных наоборот начнут соответственно расти или падать, то обучение стоит остановить, поскольку это означает, что модель начинает переобучаться.



Примечание. Можно также сказать, что до ранней остановки обучения снижается смещение, а после нее начинает увеличиваться разброс.



Остановку обучения обычно производят не сразу, а в случае если на валидационных данных ошибка растет, а accuracy соответственно падает в течение некоторого количества эпох подряд (чтобы не останавливать обучение из-за стохастических колебаний градиента/ошибки).

Такое «время ожидания» обычно обозначается через параметр **patience**.

Создадим нейросеть с оптимизатором Adam, а также реализуем в коде приведенные выше идеи валидационной выборки и ранней остановки обучения.

Нейросеть с оптимизатором Adam

Подготовка данных

Скачаем датасет MNIST.

```
(X_train, y_train), (X_test, y_test) = keras.datasets.mnist.load_data()
```

Дополнительно выделим валидационную выборку из обучающего множества.

```
X_train, X_val, y_train, y_val = train_test_split(X_train, y_train,
                                                  test_size = 0.2,
                                                  random_state = 42)
```

Приведем данные к диапазону $[-1, 1]$.

```
X_train = 2. * (X_train - np.min(X_train)) / np.ptp(X_train) - 1
X_val = 2. * (X_val - np.min(X_val)) / np.ptp(X_val) - 1
X_test = 2. * (X_test - np.min(X_test)) / np.ptp(X_test) - 1
```

Превратим изображения в векторы.

```
X_train = X_train.reshape((-1, 784)).T
X_val = X_val.reshape((-1, 784)).T
X_test = X_test.reshape((-1, 784)).T
```

Преобразуем целевую переменную с помощью one-hot encoding.

```
def ohe(y):
  examples, features = y.shape[0], len(np.unique(y))
  zeros_matrix = np.zeros((examples, features))
  for i, (row, digit) in enumerate(zip(zeros_matrix, y)):
    zeros_matrix[i][digit] = 1
```

```

  return zeros_matrix
```

```
y_train_enc, y_val_enc, y_test_enc = ohe(y_train).T, ohe(y_val).T, ohe(y_test).T
```

Посмотрим на результат.

```
X_train.shape, X_val.shape, X_test.shape
```

```
((784, 48000), (784, 12000), (784, 10000))
```

Функции активации

Объявим функции активации.

```
def sigmoid(z):
  s = 1 / (1 + np.exp(-z))
  return s

def softmax(z):
  z = z - np.max(z, axis = 0, keepdims = True)
  numerator = np.exp(z)
  denominator = np.sum(numerator, axis = 0, keepdims = True)
  softmax = numerator / denominator
  return softmax
```

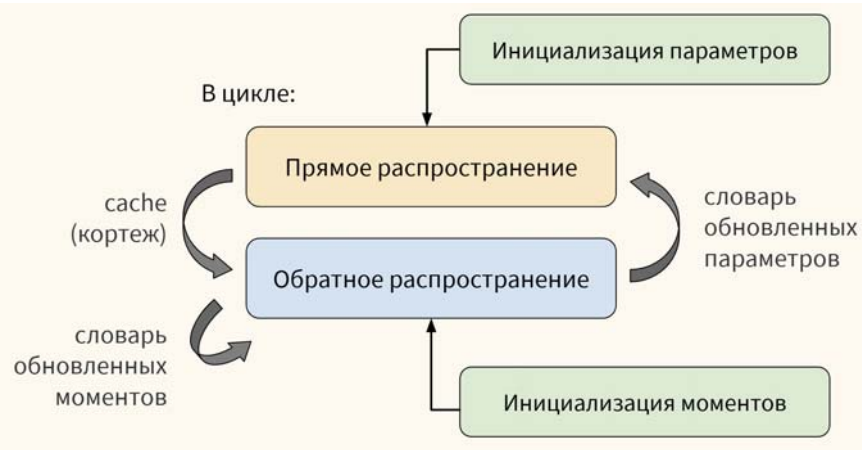
Функция потерь

И функцию потерь.

```
def cross_entropy(probs, y_enc, epsilon = 1e-9):
  n = probs.shape[1]
  ce = -np.sum(y_enc * np.log(probs + epsilon)) / n
  return ce
```

Схема нейросети

Нейросеть с оптимизатором Adam отслеживает не только обновление параметров, но и первого и второго начальных моментов.



Инициализация параметров и моментов

Параметры инициализируются случайными значениями из стандартного нормального распределения.

```
def initialize_params(x, y, random_state = 33):

  np.random.seed(random_state)

  parameters = {}

  parameters['W1'] = np.random.randn(64, x.shape[0])
  parameters['b1'] = np.random.randn(64, 1)

  parameters['W2'] = np.random.randn(64, 64)
  parameters['b2'] = np.random.randn(64, 1)

  parameters['W3'] = np.random.randn(y.shape[0], 64)
  parameters['b3'] = np.random.randn(y.shape[0], 1)

  return parameters
```

Моменты (той же размерности) инициализируются нулевыми значениями.

```
# первый начальный момент
def initialize_mk(x, y, random_state = 33):

  np.random.seed(random_state)

  mk = {}

  mk['mW1'] = np.zeros((64, x.shape[0]))
  mk['mb1'] = np.zeros((64, 1))

  mk['mW2'] = np.zeros((64, 64))
  mk['mb2'] = np.zeros((64, 1))

  mk['mW3'] = np.zeros((y.shape[0], 64))
  mk['mb3'] = np.zeros((y.shape[0], 1))

  return mk
```

```
# второй начальный момент
def initialize_vk(x, y, random_state = 33):

  np.random.seed(random_state)

  vk = {}

  vk['vW1'] = np.zeros((64, x.shape[0]))
  vk['vb1'] = np.zeros((64, 1))

  vk['vW2'] = np.zeros((64, 64))
  vk['vb2'] = np.zeros((64, 1))

  vk['vW3'] = np.zeros((y.shape[0], 64))
  vk['vb3'] = np.zeros((y.shape[0], 1))

  return vk
```

Прямое и обратное распространение

Объявим функцию прямого распространения:

- на вход эта функция будет принимать словарь параметров;
- на выход будет выдавать уроверь ошибки на обучающей выборке и кортеж параметров и значений активационных слоев (cache).

```
def forward(x, y, parameters):

  n = x.shape[1]

  W1 = parameters['W1']
  b1 = parameters['b1']
  W2 = parameters['W2']
  b2 = parameters['b2']
  W3 = parameters['W3']
  b3 = parameters['b3']

  Z1 = np.dot(W1, x) + b1
  A1 = sigmoid(Z1)

  Z2 = np.dot(W2, A1) + b2
  A2 = sigmoid(Z2)

  Z3 = np.dot(W3, A2) + b3
  A3 = softmax(Z3)

  loss = cross_entropy(A3, y)

  cache = (A1, W1, b1, A2, W2, b2, A3, W3, b3)

  return loss, cache
```

Функция обратного распространения будет:

- принимать cache и рассчитывать производные;
- принимать словари первого и второго моментов;
- обновлять текущие параметры; и наконец
- выдавать словари обновленных параметров и моментов.

```
def backward(x, y, cache, mk, vk, i, lr, beta1, beta2):

  n = x.shape[1]
  (A1, W1, b1, A2, W2, b2, A3, W3, b3) = cache
  eps = 1e-8

  W3_delta = A3 - y
  W2_delta = np.dot(W3.T, W3_delta) * A2 * (1 - A2)
  W1_delta = np.dot(W2.T, W2_delta) * A1 * (1 - A1)

  W3_derivative = np.dot(W3_delta, A2.T) / n
  b3_derivative = np.sum(W3_delta, axis = 1, keepdims = True) / n

  W2_derivative = np.dot(W2_delta, A1.T) / n
  b2_derivative = np.sum(W2_delta, axis = 1, keepdims = True) / n

  W1_derivative = np.dot(W1_delta, x.T) / n
  b1_derivative = np.sum(W1_delta, axis = 1, keepdims = True) / n

  mW1 = mk['mW1']
  mb1 = mk['mb1']
```

```
    mW2 = mk['mW2']
    mb2 = mk['mb2']
    mW3 = mk['mW3']
    mb3 = mk['mb3']

    vW1 = vk['vW1']
    vb1 = vk['vb1']
    vW2 = vk['vW2']
    vb2 = vk['vb2']
    vW3 = vk['vW3']
    vb3 = vk['vb3']

    mW3 = beta1 * mW3 + (1 - beta1) * W3_derivative
    mb3 = beta1 * mb3 + (1 - beta1) * b3_derivative
    mW2 = beta1 * mW2 + (1 - beta1) * W2_derivative
    mb2 = beta1 * mb2 + (1 - beta1) * b2_derivative
    mW1 = beta1 * mW1 + (1 - beta1) * W1_derivative
    mb1 = beta1 * mb1 + (1 - beta1) * b1_derivative

    vW3 = beta2 * vW3 + (1 - beta2) * np.square(W3_derivative)
    vb3 = beta2 * vb3 + (1 - beta2) * np.square(b3_derivative)
    vW2 = beta2 * vW2 + (1 - beta2) * np.square(W2_derivative)
    vb2 = beta2 * vb2 + (1 - beta2) * np.square(b2_derivative)
    vW1 = beta2 * vW1 + (1 - beta2) * np.square(W1_derivative)
    vb1 = beta2 * vb1 + (1 - beta2) * np.square(b1_derivative)

    mW3_hat = mW3 / (1 - beta1 ** i)
    mb3_hat = mb3 / (1 - beta1 ** i)
    mW2_hat = mW2 / (1 - beta1 ** i)
    mb2_hat = mb2 / (1 - beta1 ** i)
    mW1_hat = mW1 / (1 - beta1 ** i)
    mb1_hat = mb1 / (1 - beta1 ** i)

    vW3_hat = vW3 / (1 - beta2 ** i)
    vb3_hat = vb3 / (1 - beta2 ** i)
    vW2_hat = vW2 / (1 - beta2 ** i)
    vb2_hat = vb2 / (1 - beta2 ** i)
    vW1_hat = vW1 / (1 - beta2 ** i)
    vb1_hat = vb1 / (1 - beta2 ** i)

    mk_upd = {}

    mk_upd['mW3'] = mW3
    mk_upd['mb3'] = mb3
    mk_upd['mW2'] = mW2
    mk_upd['mb2'] = mb2
    mk_upd['mW1'] = mW1
    mk_upd['mb1'] = mb1

    vk_upd = {}

    vk_upd['vW3'] = vW3
    vk_upd['vb3'] = vb3
    vk_upd['vW2'] = vW2
    vk_upd['vb2'] = vb2
    vk_upd['vW1'] = vW1
    vk_upd['vb1'] = vb1

    parameters = {}

    parameters['W3'] = W3 - lr * mW3_hat / (np.sqrt(vW3_hat + eps))
    parameters['b3'] = b3 - lr * mb3_hat / (np.sqrt(vb3_hat + eps))
    parameters['W2'] = W2 - lr * mW2_hat / (np.sqrt(vW2_hat + eps))
    parameters['b2'] = b2 - lr * mb2_hat / (np.sqrt(vb2_hat + eps))
    parameters['W1'] = W1 - lr * mW1_hat / (np.sqrt(vW1_hat + eps))
    parameters['b1'] = b1 - lr * mb1_hat / (np.sqrt(vb1_hat + eps))
```

```

  return parameters, mk_upd, vk_upd
```

Ошибка на валидационных данных

Для того чтобы реализовать возможность ранней остановки обучения потребуется отдельная функция, которая будет выдавать уровень ошибки на валидационных данных.

```
def valuation_loss(x_val, y_val, parameters):

  W1 = parameters['W1']
  b1 = parameters['b1']
  W2 = parameters['W2']
  b2 = parameters['b2']
  W3 = parameters['W3']
  b3 = parameters['b3']

  Z1 = np.dot(W1, x_val) + b1
  A1 = sigmoid(Z1)
  Z2 = np.dot(W2, A1) + b2
  A2 = sigmoid(Z2)
  Z3 = np.dot(W3, A2) + b3
  A3 = softmax(Z3)

  val_loss = cross_entropy(A3, y_val)

  return val_loss
```

Функции fit() и predict()

Теперь соберем написанный выше код, объявив функцию **fit()** для обучения модели.

```
def fit(x, y, x_val, y_val,
        epochs, lr, batch_size = 32, random_state = 42,
        beta1 = 0.9, beta2 = 0.999,
        patience = 10, min_delta = 0):

  np.random.seed(random_state)

  x, y = x.copy(), y.copy()
  x_val, y_val = x_val.copy(), y_val.copy()

  n = x.shape[1]

  # инициализация параметров и моментов
  parameters = initialize_params(x, y)
  mk = initialize_mk(x, y)
  vk = initialize_vk(x, y)
  # степень корректировки смещения моментов
  i = 1

  # списки для записи ошибки
  # на обучающей и валидационной выборках
  loss_history, val_loss_history = [], []

  # расчет количества batches
  n_batches = n // batch_size
  if n % batch_size != 0:
    n_batches += 1

```

```
29  # наименьший достигнутый уровень ошибки
30  val_loss_so_far = float('inf')
31  # переменные наилучшей эпохи и наилучших параметров
32  best_epoch, best_parameters = None, None
33  # счетчик количества эпох без улучшения
34  patience_count = 0
35
36  # цикл эпох
37  for epoch in range(epochs):
38
39    # перемешаем индекс наблюдений
40    shuffle_idx = np.random.permutation(x.shape[1])
41    # применим новый индекс к данным
42    x_shuffled = x[: , shuffle_idx]
43    y_shuffled = y[: , shuffle_idx]
44
45    # средняя по всем batches ошибка
46    epoch_loss = 0
47
48    # цикл отдельных batches с шагом batch_size
49    for b in range(0, x.shape[1], batch_size):
50
51      # деление данных на batches
52      x_b = x_shuffled[:, b:b+batch_size]
53      y_b = y_shuffled[:, b:b+batch_size]
54
55      # обновление параметров и моментов
56      loss, cache = forward(x_b, y_b, parameters)
57      parameters, mk, vk = backward(x_b, y_b, cache, mk, vk, i,
58                                    lr, beta1, beta2)
59
60      # суммирование ошибки отдельных batches
61      epoch_loss += loss
62      # обновление степени корректировки смещения моментов
63      i += 1
64
65    # расчет и запись усредненной ошибки
66    # на обучающих данных
67    epoch_loss /= n_batches
68    loss_history.append(epoch_loss)
69
70    # расчет и запись ошибки на валидационных данных
71    val_loss = valuation_loss(x_val, y_val, parameters)
72    val_loss_history.append(val_loss)
73
74    # если разница между текущей и минимальной ошибкой
75    # на валидационных данных больше min_delta,
76    if  val_loss_so_far - val_loss > min_delta:
77      # запишем новый минимальный уровень ошибки
78      val_loss_so_far = val_loss
79      # обнулим счетик количества эпох без улучшения
80      patience_count = 0
81    else:
82      # если нет, увеличим счетчик
83      patience_count += 1
84
85    # если улучшений не будет в течение patience эпох,
86    if patience_count == patience:
87      # запишем номер эпохи и
88      best_epoch = epoch
89      # текущие параметры
90      best_parameters = parameters
91      # сообщим о ранней остановке обучения
92      # (при этом обучение продолжится для оценки уровня ошибки)
93      print(f'Early stopping at epoch {epoch}')
94      print('--------------------------------')
```

```
    if epoch % (epochs / 5) == 0:
      print('Epoch:', epoch)
      print(f'loss: {epoch_loss:.4f} - val_loss: {val_loss:.4f}')
      print('--------------------------------')

  print('Epoch:', epoch)
  print(f'loss: {epoch_loss:.4f} - val_loss: {val_loss:.4f}')

  # выведем (1) параметры в конце обучения, (2) наилучшую эпоху и
  # (3) наилучшие параметры после ранней остановки, а также
  # ошибку на (4) обучающем и (5) валидационном множествах
  return parameters, best_epoch, best_parameters, loss_history, val_loss_history
```

Также объявим функцию **predict()** для получения прогноза модели.

```
def predict(x, y, parameters):

  W1 = parameters['W1']
  b1 = parameters['b1']
  W2 = parameters['W2']
  b2 = parameters['b2']
  W3 = parameters['W3']
  b3 = parameters['b3']

  Z1 = np.dot(W1, x) + b1
  A1 = sigmoid(Z1)
  Z2 = np.dot(W2, A1) + b2
  A2 = sigmoid(Z2)
  Z3 = np.dot(W3, A2) + b3
  A3 = softmax(Z3)

  return np.argmax(A3, axis = 0)
```

Обучение и прогноз

Обучим модель и посмотрим на результат.

```
params, b_epoch, b_params, loss, val_loss = fit(X_train, y_train_enc,
                                                X_val, y_val_enc,
                                                epochs = 100,
                                                lr = 0.001,
                                                patience = 30)
```

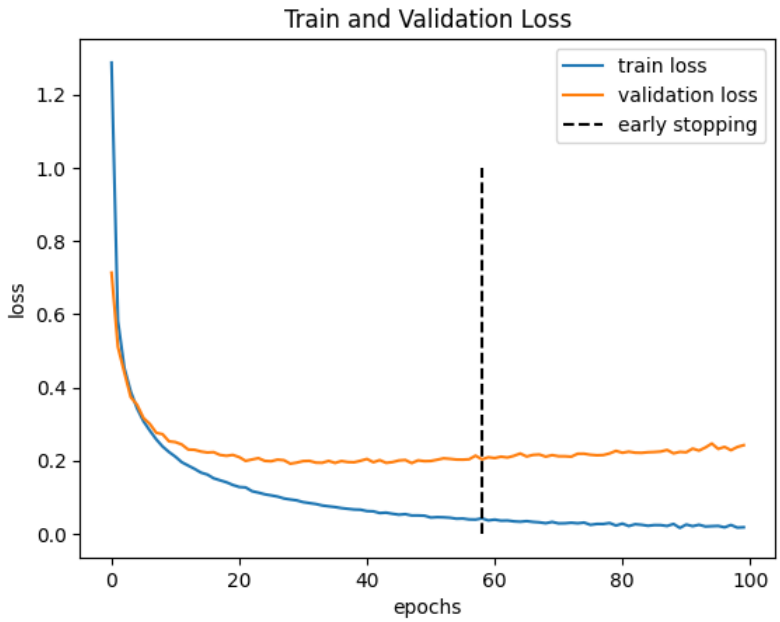
```
Epoch: 0
loss: 1.2873 - val_loss: 0.7133
--------------------------------
Epoch: 20
loss: 0.1277 - val_loss: 0.2093
--------------------------------
Epoch: 40
loss: 0.0619 - val_loss: 0.2043
--------------------------------
Early stopping at epoch 58
--------------------------------
Epoch: 60
loss: 0.0387 - val_loss: 0.2068
--------------------------------
Epoch: 80
loss: 0.0277 - val_loss: 0.2213
--------------------------------
Epoch: 99
loss: 0.0176 - val_loss: 0.2420
```

Ранняя остановка обучения произошла на эпохе 58. До этого ошибка на валидационных данных снижалась, после этого начала расти из-за того, что нейросеть начала «заучивать» данные и переобучаться.

Убедимся в этом с помощью графика.

```
plt.plot(loss, label = 'train loss')
plt.plot(val_loss, label = 'validation loss')
plt.vlines(x = b_epoch, ymin = 0, ymax = 1,
           colors = 'k', linestyles = 'dashed', label = 'early stopping')

plt.legend()
plt.title('Train and Validation Loss')
plt.xlabel('epochs')
plt.ylabel('loss')
plt.show()
```



Сделаем прогноз на обучающей выборке, а также на тестовых данных с использованием параметров после ранней остановки и параметров, достигнутых в конце обучения.

```
y_pred = predict(X_train, y_train_enc, params)
accuracy_score(y_train, y_pred)
```

```
0.9924166666666666
```

```
y_pred = predict(X_test, y_test_enc, b_params)
accuracy_score(y_test, y_pred)
```

```
0.951
```

```
y_pred = predict(X_test, y_test_enc, params)
accuracy_score(y_test, y_pred)
```

```
0.95
```

Теперь посмотрим, как эти и некоторые другие алгоритмы и инструменты можно реализовать в библиотеке Keras.

Практика с Keras

Вновь подгрузим данные.

```
(X_train, y_train), (X_test, y_test) = keras.datasets.mnist.load_data()
```

Создание модели

Объявим функцию, которая будет создавать объект класса Sequential с предобработкой данных, заданной архитектурой слоев и инициализированными параметрами. Кроме этого, сразу пропишем начальный размер шага и расписание его уменьшения.

Одновременно укажем тип оптимизатора, целевую функцию и метрику качества.

```
def create_model():

  # создадим объект класса Sequential
  model = keras.models.Sequential([
      # размерность одного наблюдения
      keras.Input(shape = (28, 28)),
      # слой для "вытягивания" картинок
      keras.layers.Flatten(),
      # приведение к диапазону от -1 до 1
      keras.layers.Rescaling(scale = 1./127.5, offset = -1),
      # полносвязные слои с соответствующим функциями активации
      keras.layers.Dense(64, activation = 'sigmoid'),
      keras.layers.Dense(64, activation = 'sigmoid'),
      keras.layers.Dense(10, activation = 'softmax')
      ])

  # объявим параметры Inverse Time Decay расписания
  initial_learning_rate = 0.001
  decay_rate, decay_steps = 0.01, 1.0

  # создадим объект класса InverseTimeDecay
  lr_schedule = keras.optimizers.schedules.InverseTimeDecay(
      initial_learning_rate,
      decay_steps, decay_rate)

  # соберем модель, прописав оптимизатор, целевую функцию и метрику качества
  model.compile(optimizer = keras.optimizers.Adam(learning_rate = lr_schedule),
                loss = 'categorical_crossentropy',
                metrics = ['accuracy'])

  return model
```

Создадим пока еще не обученную модель.

```
model = create_model()
```

Сведения о модели

Посмотрим на тип созданного объекта.

```
type(model)
```

```
keras.src.models.sequential.Sequential
def error_handler(*args, **kwargs)
```

Выведем информацию о слоях и параметрах модели.

```python
model.summary()
```

```
Model: "sequential"
┏━━━
```

Input shape: **(None, 64)**	Output shape: **(None, 10)**

Схема модели сохранится в сессионном хранилище.

Callbacks

Для того чтобы применить раннюю остановку обучения, необходимо использовать callback.

> **Callback** — это функция, которая вызывается в определенный момент в процессе обучения алгоритма.

EarlyStopping

Создадим объект класса EarlyStopping.

```
es_callback = keras.callbacks.EarlyStopping(monitor = 'val_accuracy',
                                            min_delta = 0,
                                            patience = 5,
                                            verbose = 2,
                                            restore_best_weights = True)
```

В частности, мы указали, что хотим отслеживать accuracy на валидационных данных с ожиданием без улучшения этой метрики в течение пяти эпох.

После этого мы останавливаем обучение и используем те параметры, которые показали наибольшее качество на валидационных данных.

ModelCheckpoint

Кроме этого, создадим еще один callback для сохранения весов модели. Сохраненные на диске (в сессионном хранилище) параметры можно будет загрузить в другую необученную модель с аналогичной архитектурой.

```
# сохраним параметры в файл best с расширением weights.h5
checkpoint_path = 'best.weights.h5'

cp_callback = keras.callbacks.ModelCheckpoint(filepath = checkpoint_path,
                                              save_best_only = True,
                                              save_weights_only = True,
                                              verbose = 0)
```

Созданные callbacks мы передадим в виде списка в **метод .fit()** на этапе обучения модели.

Обучение модели

Обучим модель.

```
model.fit(X_train, keras.utils.to_categorical(y_train),
          epochs = 20,
          # keras самостоятельно выделит валидационное множество
          validation_split = 0.2,
          callbacks = [es_callback, cp_callback],
          verbose = 2)
```

```
Epoch 1/20
1500/1500 - 5s - 3ms/step - accuracy: 0.7959 - loss: 1.0569 - val_accuracy: 0.8717 - v
Epoch 2/20
1500/1500 - 3s - 2ms/step - accuracy: 0.8670 - loss: 0.6497 - val_accuracy: 0.8858 - v
Epoch 3/20
1500/1500 - 3s - 2ms/step - accuracy: 0.8774 - loss: 0.5630 - val_accuracy: 0.8902 - v
Epoch 4/20
1500/1500 - 6s - 4ms/step - accuracy: 0.8827 - loss: 0.5187 - val_accuracy: 0.8928 - v
Epoch 5/20
1500/1500 - 5s - 3ms/step - accuracy: 0.8865 - loss: 0.4909 - val_accuracy: 0.8949 - v
Epoch 6/20
1500/1500 - 4s - 3ms/step - accuracy: 0.8885 - loss: 0.4712 - val_accuracy: 0.8963 - v
Epoch 7/20
1500/1500 - 5s - 3ms/step - accuracy: 0.8904 - loss: 0.4565 - val_accuracy: 0.8987 - v
Epoch 8/20
1500/1500 - 3s - 2ms/step - accuracy: 0.8925 - loss: 0.4450 - val_accuracy: 0.8992 - v
Epoch 9/20
1500/1500 - 5s - 4ms/step - accuracy: 0.8937 - loss: 0.4357 - val_accuracy: 0.9003 - v
Epoch 10/20
1500/1500 - 5s - 3ms/step - accuracy: 0.8950 - loss: 0.4279 - val_accuracy: 0.9007 - v
Epoch 11/20
1500/1500 - 5s - 3ms/step - accuracy: 0.8956 - loss: 0.4213 - val_accuracy: 0.9022 - v
Epoch 12/20
1500/1500 - 5s - 4ms/step - accuracy: 0.8965 - loss: 0.4156 - val_accuracy: 0.9030 - v
Epoch 13/20
1500/1500 - 3s - 2ms/step - accuracy: 0.8976 - loss: 0.4106 - val_accuracy: 0.9030 - v
Epoch 14/20
1500/1500 - 3s - 2ms/step - accuracy: 0.8980 - loss: 0.4061 - val_accuracy: 0.9034 - v
Epoch 15/20
1500/1500 - 3s - 2ms/step - accuracy: 0.8987 - loss: 0.4021 - val_accuracy: 0.9038 - v
Epoch 16/20
1500/1500 - 5s - 3ms/step - accuracy: 0.8989 - loss: 0.3985 - val_accuracy: 0.9039 - v
Epoch 17/20
1500/1500 - 5s - 3ms/step - accuracy: 0.8997 - loss: 0.3952 - val_accuracy: 0.9051 - v
Epoch 18/20
1500/1500 - 5s - 4ms/step - accuracy: 0.9003 - loss: 0.3922 - val_accuracy: 0.9051 - v
Epoch 19/20
1500/1500 - 5s - 3ms/step - accuracy: 0.9008 - loss: 0.3894 - val_accuracy: 0.9056 - v
Epoch 20/20
1500/1500 - 3s - 2ms/step - accuracy: 0.9013 - loss: 0.3869 - val_accuracy: 0.9058 - v
Restoring model weights from the end of the best epoch: 20.
<keras.src.callbacks.history.History at 0x7dad589018a0>
```

Как мы видим, ранняя остановка не сработала (не было пяти эпох подряд с ухудшающейся валидационной accuracy), и модель запомнила веса эпохи 20.

Данные об изменениях уровня ошибки и accuracy содержатся в словаре.

```
model.history.history.keys()
```

```
dict_keys(['accuracy', 'loss', 'val_accuracy', 'val_loss'])
```

Посмотрим, как изменялась ошибка и accuracy.

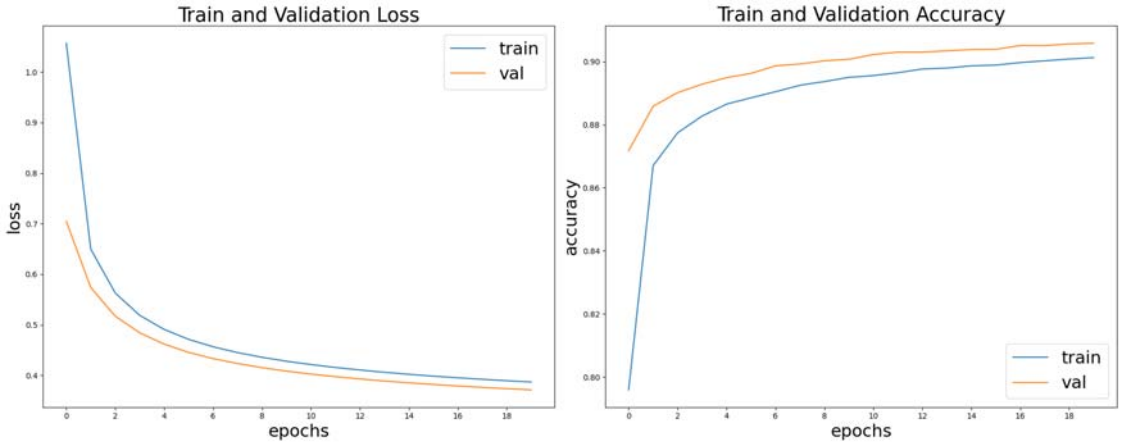
```
fig, (ax1, ax2) = plt.subplots(nrows = 1, ncols = 2,
                               figsize = (20, 8))

ax1.plot(model.history.history['loss'])
ax1.plot(model.history.history['val_loss'])
```

```
ax1.set_title('Train and Validation Loss', fontsize = 25)
ax1.set_xlabel('epochs', fontsize = 22)
ax1.set_ylabel('loss', fontsize = 22)
ax1.set_xticks(np.arange(0, 20, step = 2))
ax1.legend(['train', 'val'], loc='upper right', fontsize = 22)

ax2.plot(model.history.history['accuracy'])
ax2.plot(model.history.history['val_accuracy'])
ax2.set_title('Train and Validation Accuracy', fontsize = 25)
ax2.set_xlabel('epochs', fontsize = 22)
ax2.set_ylabel('accuracy', fontsize = 22)
ax2.set_xticks(np.arange(0, 20, step = 2))
ax2.legend(['train', 'val'], loc='lower right', fontsize = 22)

plt.tight_layout()
plt.show()
```



Оценка качества

Оценим уровень ошибки и accuracy на тестовых данных.

```
model.evaluate(X_test, keras.utils.to_categorical(y_test), verbose = 0)
```

```
[0.3715331554412842, 0.9056000113487244]
```

Восстановление модели

Создадим необученную модель.

```
new_model = create_model()
```

Сделаем прогноз на инициализированных параметрах, чтобы убедиться, что модель не обучена.

```
new_model.evaluate(X_test, keras.utils.to_categorical(y_test), verbose = 0)
```

```
[2.424980401992798, 0.09019999951124191]
```

Подгрузим сохраненные в сессионном хранилище параметры и снова сделаем прогноз.

```
new_model.load_weights(checkpoint_path)
new_model.evaluate(X_test, keras.utils.to_categorical(y_test), verbose = 0)
```

```
[0.3693920075893402, 0.906000018119812]
```

Подведем итог

В дополнительных материалах мы изучили концепцию разделения данных на обучающую, валидационную и тестовую выборки, а также идею ранней остановки обучения.

Кроме этого, мы создали нейросеть с оптимизатором Adam и инструментом ранней остановки обучения.

Наконец, мы попрактиковались в создании аналогичной нейронной сети в библиотеке Keras, применив расписание уменьшения размера шага, оптимизатор Adam, раннюю остановку обучения, а также возможность сохранения весов модели.